

CAX-Agent: A Lightweight Agent Harness for Reliable APDL Automation

Chenyang Lin^{1,6,*}, Yichen Hai^{1,2,*}, Yi He¹, Ran Wang², Haiyan Qiang², Liang Yu^{3,4,5}

¹Shanghai Ultradimension Technology Co., Ltd., Shanghai, China

²College of Logistics Engineering, Shanghai Maritime University, Shanghai, 201306, China

³School of Civil Aviation, Northwestern Polytechnical University, Xi'an 710072, China

⁴State Key Laboratory of Airliner Integration Technology and Flight Simulation, Shanghai 200126, China

⁵National Key Laboratory of Strength and Structural Integrity, Xi'an 710065, China

⁶Wuhan University, Wuhan, Hubei 430072, China

*Co-first authors with equal contribution

Abstract—Large language models deployed for MAPDL finite-element simulation face practical reliability challenges: without structured execution control, tool encapsulation, and fault recovery, outputs may be inconsistent and task failures are common. The Agent Harness paradigm addresses this by inserting domain-specific orchestration middleware that manages tool lifecycles, workflow state, and recovery escalation. This paper presents the architecture of CAX-Agent, a lightweight agent harness purpose-built for MAPDL automation, and empirically evaluates one of its core components—the recovery policy. CAX-Agent organizes execution into three layers—LLM service, agent harness, and solver backend—with a recovery ladder that escalates from deterministic rule patching through model-driven regeneration to context enrichment and human intervention. We evaluate three recovery strategies (no_recovery, rule_only, and model_only) on 50 standard structural benchmarks with three repeated runs per strategy (450 case-runs total). Two independent human raters score task completion under blind conditions; inter-rater agreement is strong (quadratic weighted Cohen’s kappa = 0.84, 96 percent of score pairs within one point). Model_only achieves the best completion rate (0.9267), task score (3.59/4), total score (9.16/10), and zero-intervention rate (0.84), outperforming rule_only (0.7733, 3.17/4, 7.03/10, 0.00) and no_recovery (0.6933, 2.74/4, 5.60/10, 0.00) with large effect sizes (Cliff’s delta = 0.81–0.87). The benchmark uses deliberately simple geometries to isolate recovery-policy effects; we discuss the scope of these findings and directions for broader validation.

Index Terms—APDL automation, finite element analysis, large language model, recovery policy, simulation reliability

I. INTRODUCTION

Computer-aided technologies are often grouped under the CAX umbrella, where CAD, CAE, and CAM represent design, engineering analysis, and manufacturing planning, respectively. In this work, the implemented pipeline enables CAD-plus-CAE automation only; CAM execution is out of scope.

LLM-driven finite-element simulation requires more than accurate code generation. The Transformer architecture established the self-attention paradigm for sequence modeling [1], and deep bidirectional pre-training extended this to representation learning [2]. Scaling to 175B parameters enabled few-shot learning without task-specific fine-tuning [3], and interleaving reasoning traces with tool-use actions improved multi-step task completion [4]. These advances have enabled

tool-using code agents, but in engineering simulation, pre-processing, solver execution, and post-processing must chain correctly, and runtime errors—meshing failures, convergence issues, missing results—are common even for structurally simple tasks. Without explicit recovery mechanisms, a single failure terminates the pipeline. As LLM-based engineering agents move toward practical use, the question of how to design and evaluate recovery policies becomes central to system reliability. CAX-Agent is designed natively for MAPDL rather than adapting a general agent framework; its recovery logic is tightly coupled to MAPDL error log syntax and APDL script structure, following a rules-first, model-second escalation strategy where deterministic rule-based patches are attempted before invoking LLM-driven repair.

The Agent Harness paradigm has emerged as the core architectural pattern for bridging this gap. Rather than expecting the LLM to manage its own execution, a harness inserts a domain-specific orchestration middleware that integrates skill encapsulation, tool orchestration, workflow checkpoints, state management, and fault diagnosis with retry escalation. This middleware provides the engineering skeleton that the LLM alone cannot supply. Analysis of 70 agent-system projects identified five recurring design dimensions—scheduler type, planning capability, recovery mechanism, context management, and implementation complexity—with Agent Loop-based schedulers remaining dominant [5]. KAIJU, an executive kernel that decouples tool execution from LLM reasoning with intent-gated execution, demonstrated that this separation enforces behavioral guarantees that prompting alone cannot match [6]. In parallel, LLM-driven agents have been applied across engineering domains. CAD automation and generative design-to-manufacturing pipelines have been explored [7, 8], alongside design structure generation and self-cognitive product design systems [9, 10]. End-to-end CFD automation with structured knowledge and reasoning has been demonstrated [11]. Broader surveys cover next-generation CAE opportunities and the manufacturing lifecycle [12, 13], as well as vision-language evaluation for engineering design and AI-empowered CAE [14, 15]; detailed discussion is deferred to Section II. These works advance agent capabilities in specific domains but do not

evaluate the recovery component of a harness under controlled, repeated conditions with human-judged outcomes.

This paper presents CAX-Agent, a lightweight, native agent harness purpose-built for APDL automation in mechanical simulation. Rather than adapting a generic harness framework, CAX-Agent is designed around the specific failure patterns observed in MAPDL execution: meshing failures, convergence errors, element-type mismatches, and missing post-processing results. Its architecture separates LLM service, harness orchestration, and solver backend into three layers, with a recovery ladder that escalates from deterministic rule patching through model-driven script regeneration to context enrichment and human intervention as a final fallback. The orchestrator—not the LLM—owns retry budgets, tool dispatch, and stop conditions.

We evaluate three recovery strategies under an identical benchmark protocol: `no_recovery` (one-shot execution), `rule_only` (deterministic rule-based patching), and `model_only` (LLM-driven error-log-conditioned regeneration with bounded retries). The benchmark uses 50 standard structural tasks—beams, plates, and cylinders under static, modal, and thermal loading—with three repeated runs per strategy (450 case-runs total). The tasks are deliberately simple. Our aim is not to push the complexity frontier of autonomous simulation, but to isolate the effect of recovery-policy design in a setting where the base task is well within the model’s capability, so that outcome differences can be attributed to the recovery policy rather than to task difficulty. We report completion behavior, multi-axis scoring (human-assessed task quality plus system-derived autonomy and efficiency), and pairwise statistical tests. `Model_only` achieves the strongest reliability while preserving high autonomy in this setting.

Our contributions are: (1) CAX-Agent, a lightweight, MAPDL-native agent harness with a three-layer architecture and recovery ladder, designed around real MAPDL failure patterns; (2) a controlled, repeated-run comparison of three recovery strategies on 50 standardized APDL tasks, with blind human scoring and inter-rater validation; and (3) empirical evidence that model-driven recovery substantially outperforms rule-based repair in both completion rate and zero-intervention rate, with per-type failure analysis showing where residual errors concentrate.

Figure 1 shows a representative end-to-end execution from the CAX-Agent interface, illustrating a conversational modal analysis task with autonomous APDL generation, MAPDL execution, and post-processing output.

II. RELATED WORK

A. LLM-Based Tool Use and Engineering Automation

Tool-using LLM agents increasingly combine reasoning traces with external actions, enabling non-trivial multi-step workflows. These capabilities are now being transferred to engineering informatics. Xu et al. substantially reduced process planning construction time using an LLM-enabled knowledge graph method [16], and Stathatos et al. framed high-level process planning as a sequence prediction task for GPT-2 in

distributed manufacturing [17]. Shi et al. fine-tuned an LLM for automated building-code compliance [18]. Wen et al. proposed an LLM-based human-machine collaborative approach for diagnosing complex industrial equipment faults [19]. Zhang et al. applied a knowledge-graph-enhanced LLM to hydraulic structure safety question answering [20], and Wang et al. applied multimodal LLMs to construction safety inspection [21]. These capabilities are directly relevant to simulation automation, where script generation must interact with strict solver interfaces and runtime feedback, as demonstrated in multi-agent aerodynamic optimization [22] and surveyed for industrial embodied intelligence [23].

In code-oriented settings, model outputs can be strong yet brittle when execution constraints are strict. Guo et al. outlined next-generation LLM-enabled CAE opportunities [12], Li et al. surveyed LLMs across the manufacturing lifecycle [13], and Picard et al. evaluated vision-language models from conceptual design through manufacturing [14]—all reporting that reliability under runtime constraints motivates explicit recovery controls in the agent loop.

B. LLM-Driven Finite Element Automation

Recent work has explored LLM-driven finite element automation from multiple angles. Mudur et al. proposed FEABench, benchmarking one-shot and agent-loop LLM capability on COMSOL multiphysics tasks and reporting that executable API call generation reaches 88% but full problem completion remains challenging [24]. Hou et al. presented AutoFEA, improving FEA input file accuracy through a GCN-Transformer retrieval model integrated with LLM planning, evaluated on CalculiX-derived benchmarks [25].

These studies advance generation quality and pipeline coverage under diverse conditions. Our work complements them by isolating recovery-policy design as a controlled variable: we keep the task set, model, and solver fixed while varying only the recovery strategy, with repeated-run statistics and multi-axis scoring. To our knowledge, no prior study reports such a controlled head-to-head comparison of recovery configurations for APDL automation.

C. Agent Execution Infrastructure

Beyond the engineering simulation domain, a parallel line of work addresses the infrastructure layer for LLM agents—the harness that manages tool lifecycles, retry logic, error propagation, and execution traces. Wei characterizes the dominant Agent Loop as a single-ready-unit scheduler and proposes Graph Harness, which separates planning, execution, and recovery into independent layers with a formalized node state machine [5]. Guerin and Guerin introduce KAIJU, an executive kernel that decouples tool execution from LLM reasoning, with Intent-Gated Execution for security and configurable execution modes for different task complexities [6]. These systems share a key design principle with CAX-Agent: the orchestrator—not the LLM—owns retry budgets, tool dispatch, and stop conditions. Where our work differs is in the empirical focus: rather than proposing a new harness architecture, we study how

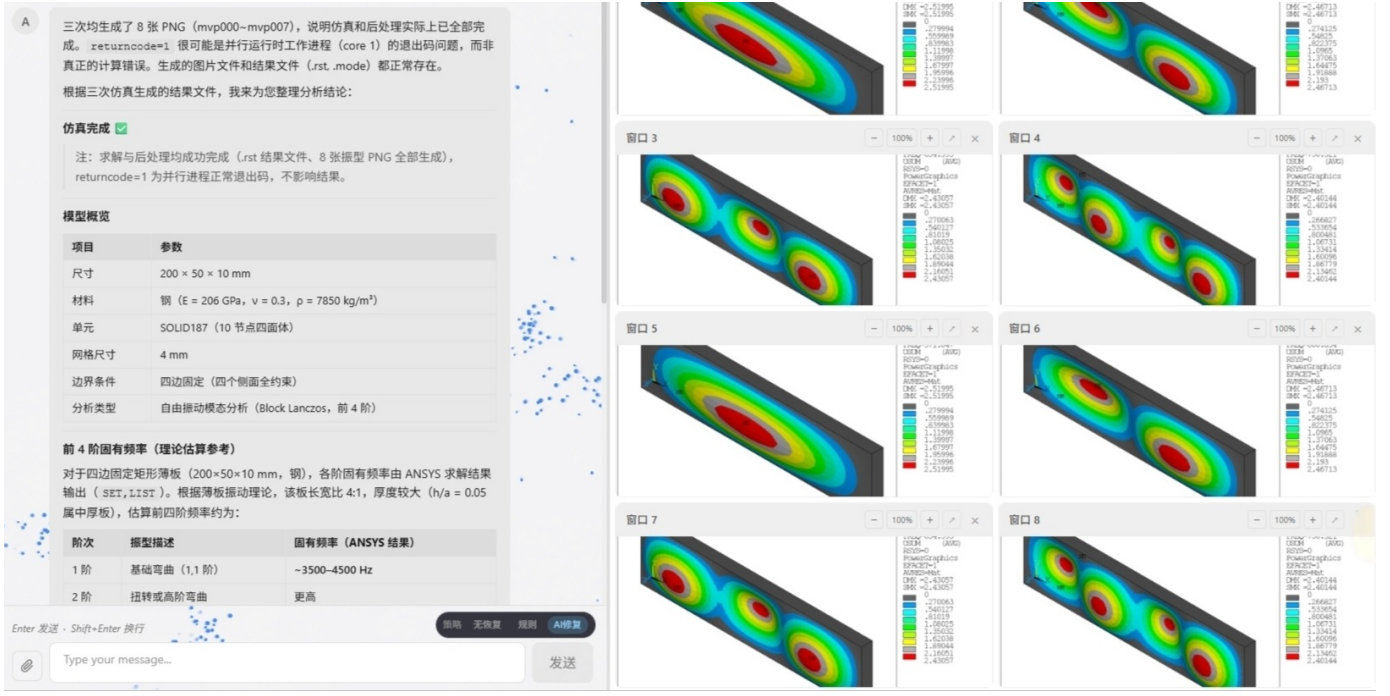


Fig. 1. End-to-end UI example from a representative modal analysis run. The system autonomously generates the APDL script, executes it in MAPDL, and produces post-processing images with a conversational interface.

a specific harness component (the recovery policy) behaves under controlled conditions with repeated measurements and human evaluation.

III. METHODOLOGY

A. System Architecture

CAX-Agent is organized as a three-layer stack for APDL-centric execution. In CAX terms, the benchmark enables CAD-plus-CAE tasks: CAD-oriented prompt interpretation and geometry/simulation script construction, followed by CAE execution and validation through MAPDL.

Layer 1 (routing layer). A FastAPI-based endpoint maintains a module registry and routes each request by module key. Incoming requests are validated against registered modules and dispatched to the corresponding sub-agent handler. This layer is responsible for function-level traffic routing across registered modules.

Layer 2 (local lightweight model layer). The runtime invokes a local inference backend for fast first-pass APDL generation and repair loop calls before returning tool actions to the orchestrator. In the deployed CAX setup, this layer runs Qwen-27B as the local model.

Layer 3 (unified external LLM API layer). External model access is unified behind a gateway configuration that manages authentication and base-URL routing. The experiment protocol fixes the external model to Claude Sonnet 4.6. This layer provides the high-capability API completion path when local reasoning is insufficient.

Above the three layers, the orchestrator converts user instructions to APDL scripts, triggers MAPDL execution,

collects logs, and coordinates bounded repair attempts. A connector layer selects available solver backends (PyMAPDL, CLI MAPDL, or fallback mode) while maintaining a unified simulation interface; retry budgets and iteration traces are recorded for post-analysis.

Figure 2 summarizes the loop. A failed execution emits solver logs that are re-injected into the model prompt for targeted regeneration. This design separates generation from execution control: the model handles semantic repair while the orchestrator enforces retry budgets and stop conditions.

B. Implementation Details

The LLM temperature was fixed at 0 throughout all experiments to eliminate stochastic variation. A typical successful case-run consumed approximately 10–15K tokens for the initial APDL generation; recovery attempts consumed additional tokens proportional to the number of retries but below the linear scaling factor of the retry count, because error-log-augmented prompts reused cached context prefixes. The full system runs on a single workstation, with the local Qwen-27B model serving fast first-pass generation and the external Claude Sonnet 4.6 API providing high-capability repair when the local model is insufficient or recovery is required. The rule set was derived from an internal engineering analysis of common MAPDL failure modes observed during system development.

C. Recovery Strategy Definitions

We compare three strategies under identical task sets and run counts:

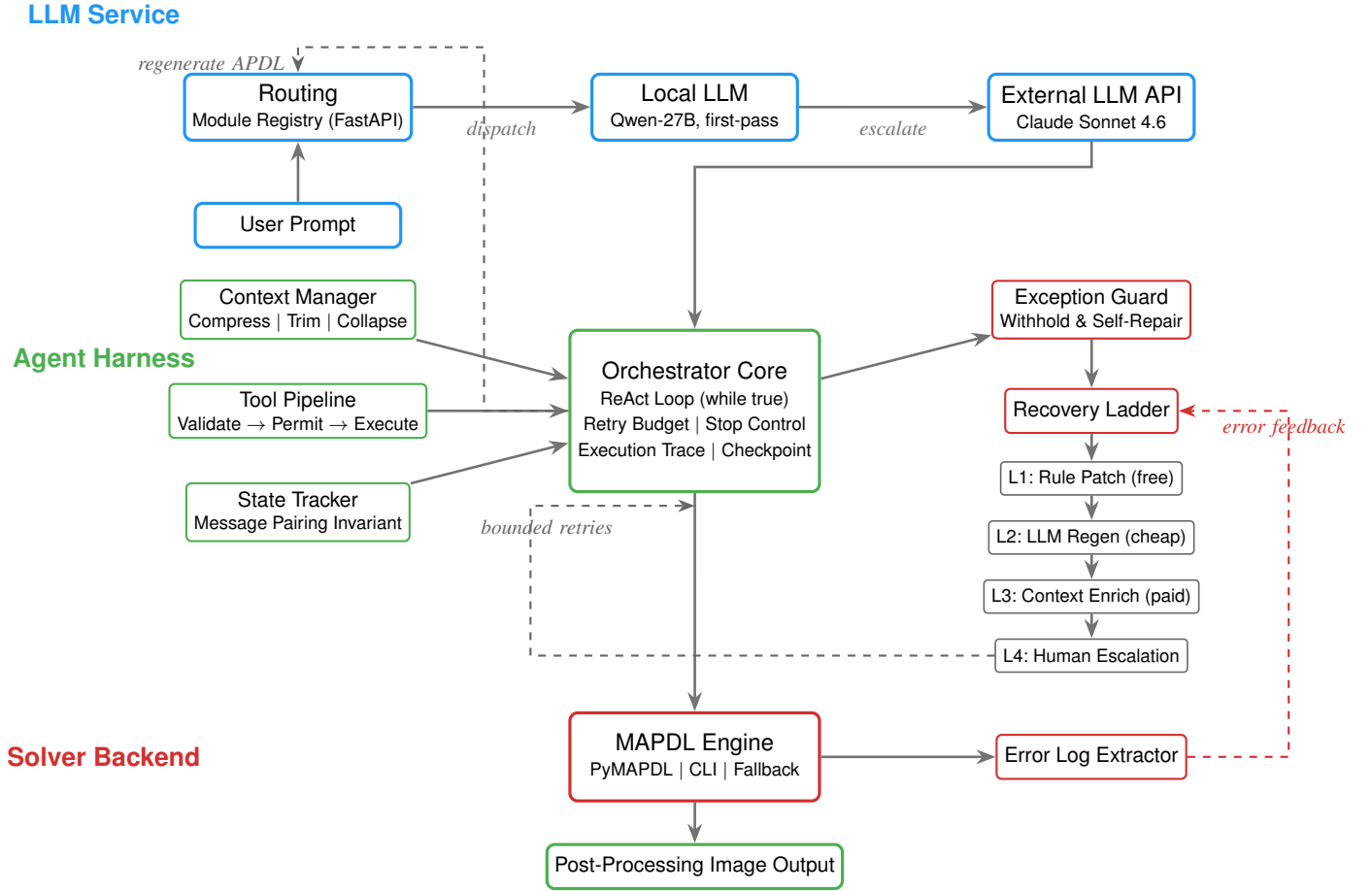


Fig. 2. CAX-Agent runtime architecture showing the three-layer harness design with recovery policy selection and feedback loops.

- **no_recovery:** one-shot execution without repair. The agent is limited to 2 ReAct iterations (the minimum required for one generate-then-execute cycle in the tool-use framework) with no forced retries, and the error-log reading tool is disabled. If the single execution fails, the task is marked as failed.
- **rule_only:** deterministic log-to-patch correction without additional model reasoning. After the initial LLM-generated APDL fails, the system reads the MAPDL error log and applies four deterministic string-transform rules: (1) mesh failure triggers element-size doubling and free-mesh fallback; (2) convergence failure inserts multi-substep and auto time-stepping directives before SOLVE; (3) element-type errors substitute compatible element formulations; (4) missing post-processing results rewrite SET commands to target the last available load step. The patched script is re-executed once. The agent performs up to 12 ReAct reasoning steps but receives no forced script retries and cannot read error logs for model-driven repair. The retry budget $B = 2$ refers to APDL execution attempts (initial plus one rule-based retry), not to ReAct reasoning steps.
- **model_only:** error-log-conditioned model regeneration

with bounded retries. The agent has access to the error-log reading tool and runs up to 12 ReAct iterations. If the model attempts to stop after a failed simulation, the orchestrator forces up to 3 additional retry rounds, instructing the model to read the error log, diagnose the failure, and regenerate the APDL script.

A task is counted as successfully completed when the pipeline produces at least one post-processing image output from MAPDL execution.

Formally, for a given case, let G_0 be the initial APDL script generated by the LLM from the user prompt. If execution of G_0 succeeds (produces an image), the case terminates. If it fails with solver error log e_1 , the recovery policy π produces a revised script $G_1 = \pi(G_0, e_1)$. In general, after the t -th failure, $G_t = \pi(G_{t-1}, e_t)$. The process repeats until either success or a strategy-specific retry budget B is exhausted:

- **no_recovery:** $B = 1$, $\pi = \emptyset$ (no repair; fail on first error).
- **rule_only:** $B = 2$, $\pi = f_{\text{rule}}(G, e)$ where f_{rule} applies up to four deterministic string-transform rules based on patterns in e .
- **model_only:** $B = 4$ (one initial attempt plus up to three forced retries), $\pi = f_{\text{LLM}}(G, e)$ where the LLM reads e

and regenerates the APDL script.

The budget B is enforced by the orchestrator, not by the LLM. This separation—the model proposes repairs, the harness enforces budgets—is the key architectural property under test.

D. Metrics and Statistical Testing

For each case-run i , we construct three scores. A human rater assigns a task completion score $t_i \in \{0, 1, 2, 3, 4\}$ by inspecting the output image for correctness and completeness. The system derives an autonomy score $a_i \in \{0, 1, 2, 3\}$ from execution logs (3 = fully autonomous, 0 = required manual intervention) and a recovery efficiency score $e_i \in \{0, 1, 2, 3\}$ from the retry count and solver outcome. The composite total score for the run is

$$q_i = t_i + a_i + e_i, \quad q_i \in [0, 10]. \quad (1)$$

Binary completion $c_i \in \{0, 1\}$ is set to 1 iff the pipeline produced at least one post-processing image. From these per-run values we compute, for each strategy s , the completion rate R_s , mean total score Q_s , and zero-intervention rate $Z_s = \frac{1}{N} \sum_i \mathbf{1}[a_i = 3]$, each aggregated over $N = 150$ case-runs.

For pairwise strategy comparisons we use Cliff's δ , a non-parametric effect size defined for two independent samples $X = \{x_1, \dots, x_n\}$ and $Y = \{y_1, \dots, y_m\}$ as

$$\delta(X, Y) = \frac{1}{nm} \sum_{i=1}^n \sum_{j=1}^m \text{sgn}(x_i - y_j), \quad (2)$$

where $\text{sgn}(z)$ is +1, -1, or 0 for $z > 0$, $z < 0$, or $z = 0$. Cliff's δ ranges from -1 to +1 and is directly interpretable: $\delta = P(X > Y) - P(X < Y)$. We follow the convention that $|\delta| < 0.147$ is negligible, < 0.33 small, < 0.474 medium, and ≥ 0.474 large. We report two-sided Mann-Whitney U p -values alongside δ , and 95% binomial confidence intervals on each completion rate.

The rule_only strategy records zero-intervention rate 0.00 because its deterministic patches require a confirmation step before re-execution, which is logged as a non-autonomous action; the strategy is rules-first with human fallback rather than automated model repair, and the zero reflects this design property, not an absence of recovery activity.

We note that case-runs sharing the same task are correlated by design (three repeats per task). As a sensitivity check, we repeated all pairwise comparisons using per-task mean scores; the between-strategy ranking and effect-size directions are unchanged. The reported results treat the 150 runs as independent observations, consistent with standard practice for controlled repeated-measure benchmarks.

Two raters independently scored t_i for all 450 case-runs under blind conditions. Inter-rater agreement is measured by quadratic weighted Cohen's κ , defined as

$$\kappa_w = \frac{p_o - p_e}{1 - p_e}, \quad (3)$$

where p_o is the observed proportion of weighted agreement and p_e is the expected proportion under chance, using squared-error weights between ordinal categories.

IV. EXPERIMENTS

A. Benchmark Scope

Our benchmark consists of 50 APDL simulation prompts spanning three categories: 35 static analysis tasks (beams, plates, brackets, pressure vessels, and simple assemblies), 10 modal analysis tasks, and 5 steady-state thermal analysis tasks. All geometries are standard structural elements with regular cross-sections and well-defined boundary conditions. The tasks are deliberately simple in their physics: they involve linear elasticity, small deformations, and single-physics setups. This design choice is intentional—by keeping the simulation domain tractable, we can attribute differences in outcomes primarily to recovery-policy design rather than to the inherent difficulty of the simulation problem. The benchmark is not intended to represent the full complexity of industrial CAE workflows; it is a testbed for comparing agent recovery behavior under controlled conditions.

B. Benchmark Protocol

Two raters independently scored task completion under blind conditions. Quadratic weighted Cohen's $\kappa = 0.84$, indicating excellent agreement; 96% of score pairs fall within one point. Both raters produce the same strategy ranking (model_only > rule_only > no_recovery). Rater B's means are systematically 0.3–0.5 points lower, reflecting a stricter threshold, but between-strategy gaps remain consistent. All reported results use Rater A scores; Rater B statistics confirm the same conclusions. The per-case scoring data is available from the authors upon reasonable request.

C. Main Results

Table I reports the core metrics. Model_only leads on all indicators: completion rate 0.9267 (95% CI: 0.885–0.968), task score 3.59/4, total score 9.16/10, and zero-intervention 0.84. Completion rates for the baselines are substantially lower: 0.7733 (95% CI: 0.706–0.840) for rule_only and 0.6933 (95% CI: 0.620–0.767) for no_recovery; the confidence intervals for model_only do not overlap with those of either baseline.

Pairwise effect sizes, measured by Cliff's δ on the total score distribution, are large: $\delta = 0.81$ for model_only vs. rule_only, $\delta = 0.87$ for model_only vs. no_recovery, and $\delta = 0.57$ for rule_only vs. no_recovery (all $p < 0.001$, Mann-Whitney U). An effect of this magnitude means that a randomly chosen model_only case-run outscores a rule_only case-run with probability $(0.81 + 1)/2 \approx 0.91$.

On the human-scored task completion axis alone, model_only outperforms rule_only by +0.42 points and no_recovery by +0.85 points (Mann-Whitney U, $p = 2.74 \times 10^{-3}$ and $p = 1.85 \times 10^{-4}$). Rule_only vs. no_recovery on task score is not significant ($p = 0.173$), indicating that deterministic rule patching improves system-level autonomy metrics but does not reliably raise output quality as judged by human raters.

Figure 3 visualizes these results across the three primary metrics, highlighting the consistent lead of model_only.

TABLE I

OVERALL BENCHMARK RESULTS (150 CASE-RUNS PER STRATEGY). TASK SCORE IS HUMAN-ASSESSED; AUTONOMY AND EFFICIENCY ARE SYSTEM-DERIVED AND FOLDED INTO THE TOTAL.

Strategy	Compl. Rate	Task (/4)	Total (/10)	Zero-Interv.
model_only	0.9267	3.5867	9.1600	0.8400
rule_only	0.7733	3.1667	7.0267	0.0000
no_recovery	0.6933	2.7400	5.6000	0.0000

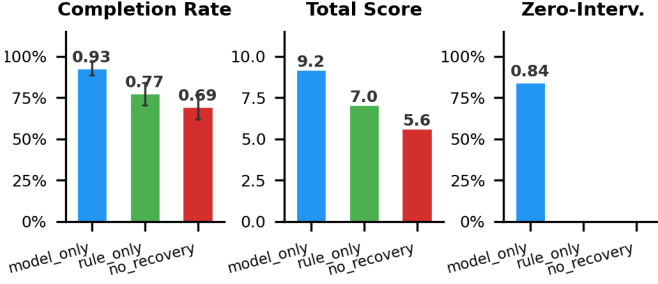


Fig. 3. Comparison of completion rate, scaled average total score, and zero-intervention rate across the three recovery strategies. The model_only strategy consistently leads all metrics.

D. Per-Type Analysis and Robustness

The task-type breakdown in Figure 4 shows that model_only maintains the top completion and score levels on static, modal, and thermal subsets. The largest margin appears in thermal tasks, where no_recovery drops to 0.5333 completion while model_only stays at 0.9333.

Score distribution analysis in Figure 5 further indicates that model_only is more robust: its quartiles are concentrated near the top of the 0–10 range, whereas no_recovery shows a lower center and wider degradation.

Majority-case completion rates (across three repeats) are 0.94 for model_only, 0.80 for rule_only, and 0.72 for no_recovery. Persistent hard-case IDs are concentrated in rule_only and no_recovery, suggesting deterministic edits alone are insufficient for long-tail solver and script-path failures.

E. Failure-Case Analysis and Discussion

Failure distribution is not uniform across simulation categories. At majority-case level, model_only has 3 failed cases, all in static analysis (3 static, 0 thermal, 0 modal). Rule_only has 10 failed cases (9 static, 1 thermal, 0 modal). no_recovery has 14 failed cases (12 static, 2 thermal, 0 modal). This indicates that most residual failures are concentrated in static structural settings.

For model_only, the remaining failed static cases are concentrated in case IDs 8, 21, and 35. Combined with manual inspection, these failures are more correlated with thin-wall or mesh-sensitive geometric features than with nominal task complexity. In other words, some multi-part assemblies can still be solved successfully, while certain thin-wall geometries remain brittle due to mesh handling quality. Figure 6 illustrates the mesh re-partitioning and re-meshing pipeline proposed to address such geometry-driven failures.

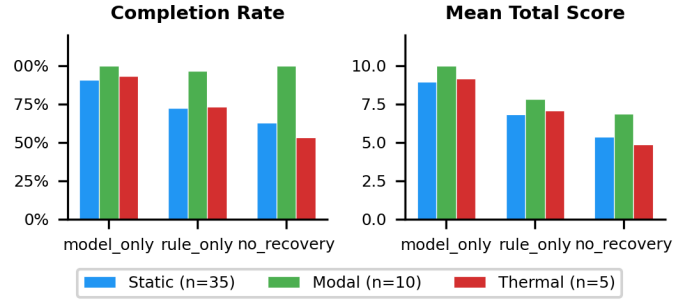


Fig. 4. Task-type breakdown over static, modal, and thermal subsets. Model_only keeps the strongest completion and score levels across all subsets, with the largest margin on thermal tasks.

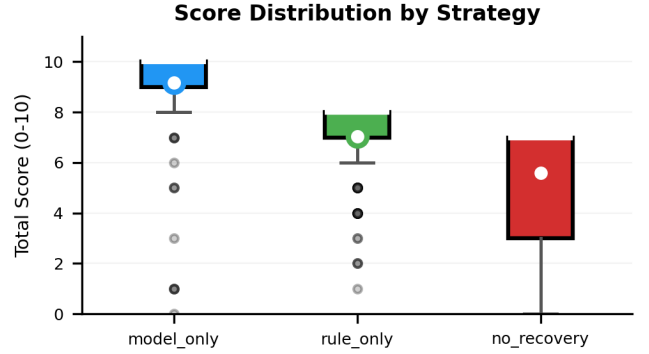


Fig. 5. Interquartile score ranges and medians for each strategy. Model_only concentrates near the top of the 0–10 scale, while no_recovery shows lower and wider central tendency.

The empirical pattern supports a simple interpretation: deterministic rules are useful but bounded by pre-specified templates, while model-driven repair adapts to broader failure signatures. Importantly, model_only improves both completion and autonomy, indicating that gains are not achieved by shifting burden to human operators. For current CAX deployment, this places model_only as the most practical default strategy.

V. LIMITATIONS

Several limitations of this study should be noted. First, the benchmark uses 50 structurally simple geometries (beams, plates, and cylinders with regular cross-sections). It does not include complex assemblies, nonlinear material models, large-deformation analysis, or multiphysics coupling. Results may not transfer directly to these more challenging settings. Second, the experiment is confined to a single solver backend (MAPDL) and a single external model (Claude Sonnet 4.6). Recovery behavior may differ with other solver interfaces or model versions. Third, the rule set used in rule_only was derived from an internal engineering analysis of MAPDL failure modes rather than a systematic, exhaustive taxonomy; its effectiveness is bounded by the coverage of the failure patterns analyzed. Fourth, we report three repeated runs per strategy; larger repetition counts would yield tighter confidence intervals and more robust failure-mode statistics. Fifth, this study isolates the recovery policy as the experimental variable;

TABLE II
FAILED-CASE DISTRIBUTION BY STRATEGY AND SIMULATION TYPE
(MAJORITY-CASE CRITERION).

Strategy	Total Failed	Static	Thermal	Modal
model_only	3	3	0	0
rule_only	10	9	1	0
no_recovery	14	12	2	0

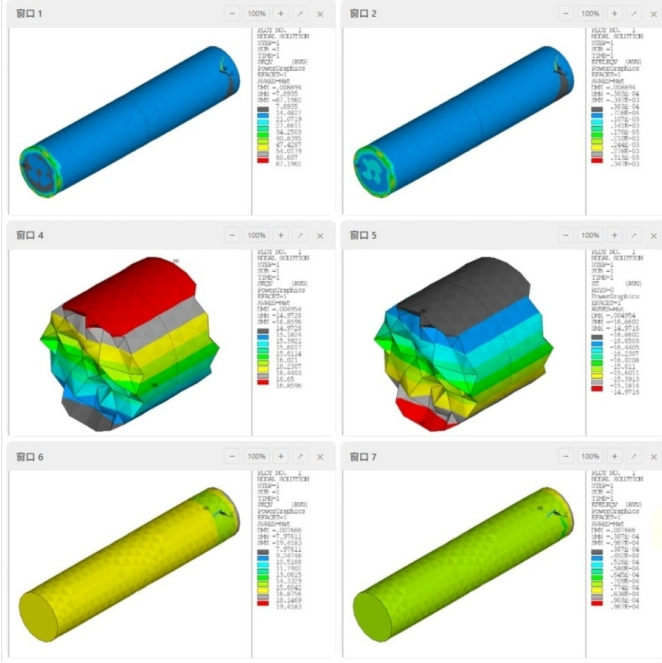


Fig. 6. Mesh re-partition and re-meshing workflow. The visual pipeline indicates improved mesh quality after reprocessing, which is critical for reducing mesh-driven solver failures.

other harness components shown in the architecture (Context Manager, Tool Pipeline, State Tracker) are part of the system design but are not individually ablated. Their contributions remain to be quantified in future work. Sixth, we do not directly compare CAX-Agent against FEABench, AutoFEA, or other recent FEA automation systems because these systems target different solver backends and task distributions; cross-system comparison on a common benchmark is an important direction for future work. Seventh, the recovery ladder shares conceptual similarity with the three-level hierarchy recently formalized in the Dual-State Action Pair framework [26]. While this parallel supports the generality of layered recovery in agent harnesses, the present study does not provide a head-to-head comparison with Thompson’s framework, and both that work and the harness-theory references [5, 6] are currently available only as arXiv preprints; their conclusions should be interpreted with appropriate caution pending peer-reviewed publication.

These limitations bound the scope of our conclusions but do not invalidate the core finding: under controlled conditions, model-driven recovery outperforms both no-recovery and rule-based repair on the metrics reported.

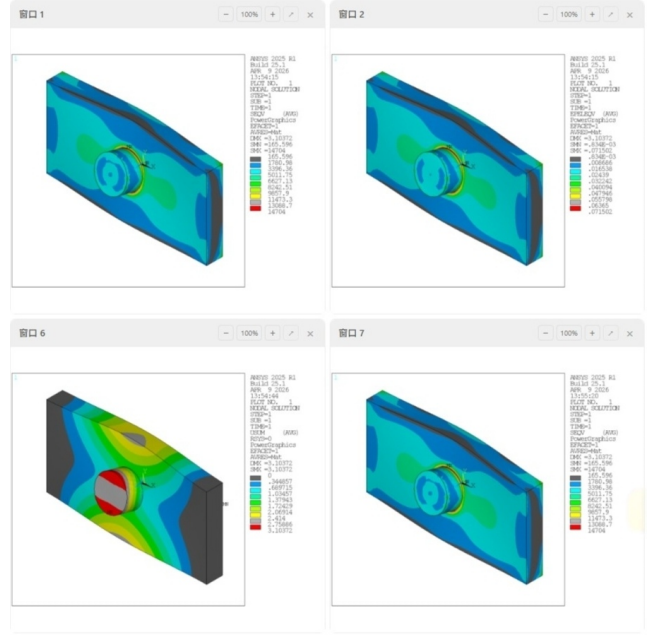


Fig. 7. Static pin-joint case ($100 \times 50 \times 10$ plate with a $\phi 20$ hole and $\phi 20$ pin under 2000N lateral load). This example illustrates that failure is tied to geometry-meshing sensitivity rather than simply the number of parts in the assembly.

VI. CONCLUSION

This paper presented CAX-Agent, a lightweight agent harness for MAPDL-based finite-element automation, and evaluated its recovery component through a controlled comparison of three strategies on 50 standardized structural benchmarks with repeated runs, blind human scoring, and inter-rater validation. Model_only achieved the best results across all metrics: completion rate 0.9267, task score 3.59/4, total score 9.16/10, and zero-intervention rate 0.84, with large and statistically significant pairwise gains over rule_only and no_recovery. The zero-intervention gap between model_only (0.84) and rule_only (0.00) is particularly notable: deterministic rules, while improving completion over no recovery, never reached fully autonomous operation, underscoring the value of model-driven recovery within a harness architecture.

CAX-Agent demonstrates that a lightweight, domain-native harness—rather than a generic multi-agent framework—can transform MAPDL simulation from scattered, unreliable LLM calls into a standardized, traceable, and repeatable engineering workflow. The three-layer design, the recovery ladder, and the orchestrator-centric control model address the key failure modes that prevent reliable single-model deployment in mechanical simulation.

The benchmark scope is deliberately narrow: simple geometries, linear elasticity, single-physics. Validation on larger and more diverse benchmarks, across multiple solver backends and model versions, is needed before practical deployment. Extending recovery into the pre-processing stage—through adaptive meshing or geometry-aware partitioning—may address the mesh-sensitive failures that persist under script-level repair.

From a harness-engineering perspective, the recovery-policy evaluation methodology used here—controlled ablation, repeated runs, multi-axis scoring with inter-rater validation—can be applied to other harness components and other simulation backends, offering a reproducible template for empirical harness evaluation as the field matures.

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 2019.
- [3] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.
- [5] H. Wei, "From agent loops to structured graphs: A scheduler-theoretic framework for LLM agent execution," *arXiv preprint arXiv:2604.11378*, 2026.
- [6] C. Guerin and F. Guerin, "KAJU: An executive kernel for intent-gated execution of LLM agents," *arXiv preprint arXiv:2604.02375*, 2026.
- [7] A. Daareyni, A. Martikkala, H. Mokhtarian, and I. F. Ituarte, "Generative AI meets CAD: Enhancing engineering design to manufacturing processes with large language models," *The International Journal of Advanced Manufacturing Technology*, Jun. 2025.
- [8] H. Deng, S. Khan, and J. A. Erkoyuncu, "An investigation on utilizing large language model for industrial computer-aided design automation," *Procedia CIRP*, vol. 128, pp. 221–226, 2024.
- [9] E. C. Koh, "Using a large language model to generate a design structure matrix," *Natural Language Processing Journal*, vol. 9, p. 100103, Dec. 2024.
- [10] X. Liang, Z. Wang, and J. Liu, "Towards a self-cognitive complex product design system: A fine-grained multi-modal feature recognition and semantic understanding approach using large language models in mechanical engineering," *Advanced Engineering Informatics*, vol. 65, p. 103265, May 2025.
- [11] E. Fan, K. Hu, Z. Wu, J. Ge, J. Miao, Y. Zhang, H. Sun, W. Wang, and T. Zhang, "ChatCFD: A large language model-driven agent for end-to-end computational fluid dynamics automation with structured knowledge and reasoning," *Advanced Intelligent Discovery*, 2025.
- [12] J. Guo, C. Park, D. Qian, T. J. Hughes, and W. K. Liu, "Large language model-empowered next-generation computer-aided engineering," *Computer Methods in Applied Mechanics and Engineering*, vol. 450, p. 118591, Mar. 2026.
- [13] Y. Li, H. Zhao, H. Jiang, Y. Pan, Z. Liu, Z. Wu, P. Shu, J. Tian, T. Yang, S. Xu, Y. Lyu, P. Blenk, J. Pence, J. Rupram, E. Banu, K. Song, D. Zhu, X. Wang, and T. Liu, "Large language models for manufacturing," *Journal of Manufacturing Systems*, vol. 86, pp. 516–545, Jun. 2026.
- [14] C. Picard, K. M. Edwards, A. C. Doris, B. Man, G. Giannone, M. F. Alam, and F. Ahmed, "From concept to manufacturing: Evaluating vision-language models for engineering design," *Artificial Intelligence Review*, vol. 58, no. 9, Jul. 2025.
- [15] X. Zhao, X.-M. Tong, F. Ning, M.-L. Cai, F. Han, and H. Li, "Review of empowering computer-aided engineering with artificial intelligence," *Advances in Manufacturing*, vol. 14, pp. 103–143, 2025.
- [16] Q. Xu, F. Qiu, G. Zhou, C. Zhang, K. Ding, F. Chang, F. Lu, Y. Yu, D. Ma, and J. Liu, "A large language model-enabled machining process knowledge graph construction method for intelligent process planning," *Advanced Engineering Informatics*, vol. 65, p. 103244, May 2025.
- [17] E. Stathatos, P. Benardos, G.-C. Vosniakos, D. Gross, H. Spieker, and A. Gotlieb, "Large language models for high-level computer-aided process planning in a distributed manufacturing paradigm," *Robotics and Computer-Integrated Manufacturing*, vol. 100, p. 103233, Aug. 2026.
- [18] J. Shi, W. Solihin, and J. K. W. Yeoh, "Fine-tuning a large language model for automated code compliance of building regulations," *Advanced Engineering Informatics*, vol. 68, p. 103676, Nov. 2025.
- [19] S. Wen, F. Li, W. Zhuang, X. Pan, W. Yu, J. Bao, and X. Li, "Leveraging large language models for human-machine collaborative troubleshooting of complex industrial equipment faults," *Advanced Engineering Informatics*, vol. 65, p. 103235, May 2025.
- [20] D. Zhang, G. Ma, T. Qu, X. Wang, W. Zhou, and X. Wang, "A knowledge graph-enhanced large language model for question answering of hydraulic structure safety management," *Advanced Engineering Informatics*, vol. 66, p. 103468, Jul. 2025.
- [21] Y. Wang, H. Luo, and W. Fang, "An integrated approach for automatic safety inspection in construction: Domain knowledge with multimodal large language model," *Advanced Engineering Informatics*, vol. 65, p. 103246, May 2025.
- [22] Y. Fan, H. Zhan, M. Zhang, and B. Mi, "AirfoilAgent: Airfoil aerodynamics optimization design via large language model multi-agent collaborations," *Advanced Engineering Informatics*, vol. 71, p. 104246, Apr. 2026.
- [23] J. Zhu *et al.*, "A review on large language models for industrial embodied intelligence," *Advanced Engineering Informatics*, vol. 73, p. 104602, Jul. 2026.
- [24] N. Mudur, H. Cui, S. Venugopalan, P. Raccuglia, M. P. Brenner, and P. Norgaard, "FEABench: Evaluating language models on multiphysics reasoning ability," in *NeurIPS 2024 Workshop on Mathematical Reasoning and AI (MATH-AI)*, 2024, arXiv:2504.06260.
- [25] S. Hou, R. Johnson, R. Makhija, L. Chen, and Y. Ye, "AutoFEA: Enhancing AI copilot by integrating finite element analysis using large language models with graph neural networks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 22, 2025.
- [26] M. Thompson, "The dual-state architecture for reliable LLM agents," *arXiv preprint arXiv:2512.20660*, 2026.